

FDE_USER_41

PRIYANKA KARUPPUSAMY

USE CASE NUMBER: UC007

Claims Processing Workspace

1. Problem Statement

Insurance claims teams receive evidence from repair estimates, damage photographs, incident and police reports, customer statements, policy documents and claim forms. Manual review across these sources is slow and inconsistent, making it difficult to detect missing evidence, reconcile conflicting details and prepare an accurate claim view for specialist decision-making.

The opportunity is to build a Claims Processing Agent that orchestrates claim intake and review in one auditable workflow. The agent classifies documents, extracts claim facts, checks completeness and policy alignment, compares evidence across sources, flags discrepancies or suspicious patterns, and produces grounded summaries and recommendations while keeping final decisions with authorized claims professionals.

2. Delivery Scope

The implementation has two delivery parts.

Part 1: Capstone-provided work

Part 1 contains both the base lab and the selected extension challenges.

2.1 Base lab implementation from docs/lab-guide.md

The completed base implementation includes:

Area	Files	Why this approach was used
Claim-form schema	backend/analyzers.py	Content Understanding provides typed fields and confidence rather than relying on unstructured model text.
Photograph assessment	backend/vision.py	A vision model describes only visible damage and maps it to a conservative indicative repair band.
Deterministic validation	backend/validation.py	Repeatable rules make findings explainable, testable, and easy for a handler to verify.
Grounded agent	backend/claims_agent.py	The agent retrieves policy guidance through the existing MCP Knowledge

Area	Files	Why this approach was used
		Base and returns a strict JSON review shape.
Orchestration	backend/pipeline.py	A single flow coordinates ingest, classification, extraction, vision, validation, and summarisation.
Policy configuration	backend/config.py	Environment values and business thresholds are centralized.
Evaluation	eval/evaluate.py, data/ground-truth.json	The original five claims are scored for extraction, findings, recommendations, and safety.

The deterministic rules intentionally remain separate from the language model. The model explains the result; it does not replace the rule engine for dates, identifiers, completeness, authority, or evidence consistency.

2.2 Extension F1: hire-car invoice evidence

Implemented changes:

File	Change
backend/analyzers.py	Added HIRE_CAR_INVOICE fields for invoice reference, vehicle registration, hire dates, daily rate, days, and total amount; registered filename classification for hire-car-invoice and hire_car_invoice.
backend/validation.py	Added hire-period checks for a start date before loss, an end date after expected repair completion, excessive duration, invalid date order, and date/day-count mismatch.
backend/config.py	Added hire-invoice limits and critical-field configuration.
data/claims/CLM-2026-0431/hire-car-invoice.pdf	Added the supplied hire-car evidence.
data/ground-truth.json	Added the invoice expectations using the actual PDF values.

File	Change
eval/evaluate.py	Extended extraction and finding scoring for F1 expectations.
tests/test_extensions.py	Added classification, valid-boundary, before-loss, after-repair, and currency tests.

2.3 Extension F5: low-confidence verification queue

Implemented changes:

File	Change
backend/verification.py	Selects low-confidence fields that materially affect rules or outcomes, creates stable verification IDs, and removes duplicates.
backend/config.py	Defines the confidence threshold and critical fields by document type.
backend/pipeline.py	Adds verification items and an awaiting_verification status to the prepared result.
backend/store.py	Persists verification items and confirmations.
backend/api.py	Accepts confirmation IDs as part of a named handler decision.
frontend/src/components/ClaimDetail.jsx	Displays the field, extracted value, source document, and confidence.
frontend/src/components/DecisionPanel.jsx	Requires every verification item to be confirmed before a decision is recorded.
tests/test_extensions.py	Tests that only critical low-confidence fields are routed to verification.

The design avoids burdening the handler with every low-confidence field. A low-confidence policy number, loss date, VIN, estimate, or damage area matters more than a low-confidence descriptive field.

2.4 Extension F7: batch and operational metrics

F7 adds a manager-level view across processed claims.

Implemented changes:

File	Change
backend/store.py	Aggregates recommendation mix, finding codes, missing documents, estimate values, preparation time, waiting-on-policyholder claims, senior-adjuster referrals, and claims cleared without a warning or critical finding. Each metric includes claim IDs for traceability.
backend/api.py	Exposes GET /api/metrics.
frontend/src/components/MetricsOverview.jsx	Displays operational metrics before claim detail and lets the manager select an underlying claim.
frontend/src/App.jsx	Loads and refreshes metrics with the claim queue.
frontend/src/styles.css	Styles the metrics overview and responsive layout.
extensions/F7-batch-metrics.md	Defines the challenge and acceptance criteria.

The dashboard reports operational states such as waiting on the policyholder and needing a senior adjuster, rather than only technical counts.

3. Part 2: Customizations Added Beyond the Capstone Extensions

3.1 Human decision queue and lifecycle sections

The workspace now separates claims into collapsible sections:

Awaiting handler decision, Accepted preparations, Request information, Rejected preparations, Archive, Potential duplicates awaiting handler review, Confirmed duplicate claims.

The handler actions are explicit:

Accept preparation, Request information, Reject preparation, Mark as duplicate when a potential match is detected, Edit a completed decision, Archive a completed decision.

Decision history, handler name, note, timestamp, and verification confirmations are persisted in Azure Table Storage. Reprocessing or uploading evidence for a request-information claim returns the claim to the active preparation queue.

Files involved:

backend/store.py, backend/api.py, frontend/src/components/ClaimList.jsx, frontend/src/components/ClaimDetail.jsx, frontend/src/components/DecisionPanel.jsx,, frontend/src/App.jsx, frontend/src/styles.css

3.2 Evidence upload and claim submission

POST /api/claims/submit accepts multiple PDF or image files, validates the claim ID and file types, enforces upload limits, stores the files under the claim evidence folder, and starts the normal pipeline.

The UI component is frontend/src/components/UploadClaim.jsx.

The same claim ID is allowed only when the claim is explicitly waiting for information. Other existing IDs return HTTP 409 to prevent accidental overwrites and duplicate claim-ID creation.

3.3 Duplicate claim detection

Duplicate identity uses a conservative strong identity:

Normalized policy number, Normalized VIN, Date of loss

A new claim ID matching all three is placed in duplicate_pending. This is not a final decision. The handler sees the matching claim ID and evidence, then chooses duplicate, accept preparation, request information, or reject preparation.

The confirmed duplicate state is separate from the system-detected pending state. This preserves the human-in-the-loop boundary.

Files involved:

backend/store.py, backend/api.py, frontend/src/components/ClaimList.jsx, frontend/src/components/ClaimDetail.jsx, frontend/src/components/DecisionPanel.jsx, tests/test_extensions.py

3.4 Processing feedback and reprocessing

The API returns active background claim IDs in the claims response. The frontend uses that list as the source of truth, so a claim remains visibly in processing even before the prepared record is saved.

The UI now:

Shows a backend-processing banner, Disables repeated Process clicks, Shows Process again for eligible processed claims., Polls while work is active, Shows Processed and ready for handler review after completion.

Files involved:

frontend/src/App.jsx, frontend/src/components/ClaimList.jsx, backend/api.py

3.5 Performance improvements

The health endpoint no longer scans every claim-status partition simply to render the header. This reduced the measured health request from several seconds to approximately tens of milliseconds in the lab environment.

The UI also gives immediate feedback while the slower Azure extraction, vision, and grounded-agent calls continue.

Files involved:

backend/api.py, frontend/src/App.jsx, frontend/src/styles.css

3.6 Input validation and safety hardening

Implemented protections include:

Claim ID validation, Safe evidence-path resolution with Path.is_relative_to().

Upload suffix, count, and size limits, Named-handler validation.

Decision validation through Pydantic, Decimal parsing for currency thresholds.

Duplicate finding suppression, Bounded policy-schedule extraction retry.

Malformed PDF/photo handling with logged failures, Safe reset utility requiring --yes.

Files involved:

backend/api.py, backend/config.py, backend/pipeline.py, backend/store.py, backend/validation.py, backend/clear_claim_records.py

3.7 Testing and verification

Added focused tests in tests/test_extensions.py covering:

Hire-invoice classification, Hire-period boundaries.

Exact Decimal authority thresholds, Critical low-confidence verification routing.

Duplicate identity requirements.

Validation commands:

From the repository root

.\env\Scripts\python.exe -m unittest discover -s tests -v

cd frontend

npm run build

cd ..\backend

```
..\env\Scripts\python.exe -m py_compile *.py
```

```
cd ..\eval
```

```
python evaluate.py
```

The last complete Azure-backed evaluation run before clearing records passed:

- Extraction: 50/50
- Findings: 10/10
- Recommendations: 5/5
- Safety violations: 0
- Claims fully passed: 5/5

4. Why the Architecture Uses These Choices

Deterministic rules plus grounded generation

Rules are used where repeatability matters. The agent is used for concise explanation, policy citation, outstanding items, and next steps. This reduces hallucination risk and makes findings independently testable.

Existing Knowledge Base connection

The application connects to the existing policy Knowledge Base through MCP. It does not implement its own policy chunking or indexing. Knowledge Base ingestion, retrieval preparation, and chunking are managed by the existing Azure service.

Azure Table persistence

Azure Table Storage provides a simple durable store for prepared claim packages and handler decisions. Large JSON values are split into smaller table properties by store.py because of property-size limits. This storage chunking is unrelated to Knowledge Base document chunking.

Human-in-the-loop decision boundary

Every claim must reach a named handler. Low-confidence fields require confirmation, duplicate matches require human confirmation, fraud indicators cause referral rather than accusation, and the system never approves, declines, or pays a claim.

5. Source File Overview

Backend

File	Responsibility and flow position
backend/config.py	Loads environment values, service endpoints, aliases, business limits, confidence thresholds, upload limits, and table names.

File	Responsibility and flow position
backend/analyzers.py	Defines Content Understanding schemas and maps filenames to document types.
backend/content_understanding.py	Calls Azure Content Understanding analyzers, polls operations, and flattens typed fields.
backend/vision.py	Encodes photographs, calls the vision model, and returns structured visible-damage assessments.
backend/validation.py	Runs deterministic completeness, consistency, coverage, authority, estimate, repairer, frequency, confidence, and hire-invoice rules.
backend/verification.py	Selects critical low-confidence fields for handler confirmation.
backend/claims_agent.py	Connects to the existing policy KB through MCP and creates grounded structured reviews.
backend/pipeline.py	Orchestrates evidence ingest, extraction, photo assessment, history parsing, validation, duplicate preparation, and review generation.
backend/store.py	Persists claim packages, status partitions, decisions, verification confirmations, metrics, duplicate identity checks, and archive transitions.
backend/api.py	Exposes health, claims, evidence, file, process, upload, decision, archive, and metrics endpoints.
backend/clear_claim_records.py	Safely clears persisted Azure claim records with dry-run behavior and optional output cleanup.

Frontend

File	Responsibility and flow position
frontend/src/App.jsx	Loads the queue, metrics, health state, upload form, active processing state, and selected detail.

File	Responsibility and flow position
frontend/src/components/ClaimList.jsx	Renders collapsible lifecycle sections, process/reprocess actions, upload actions, and duplicate labels.
frontend/src/components/ClaimDetail.jsx	Displays summary, duplicate context, findings, verification, extracted fields, photos, evidence, citations, and decisions.
frontend/src/components/DecisionPanel.jsx	Records handler outcomes, verification confirmations, edits, and archive requests.
frontend/src/components/MetricsOverview.jsx	Displays traceable operational metrics.
frontend/src/components/UploadClaim.jsx	Submits new evidence and missing documents through multipart upload.
frontend/src/styles.css	Provides the workspace layout, statuses, findings, metrics, upload, and responsive styles.
frontend/vite.config.js	Runs Vite and proxies /api calls to the backend port.

Data, policy, and evaluation

Path	Responsibility
data/claims/	Sample and user-uploaded evidence folders.
data/ground-truth.json	Expected extraction, findings, recommendations, and F1 extension facts.
data/photo-credits.json	Photo provenance metadata.
reference/policies/	Policy corpus used by deterministic rules and the existing KB.
eval/evaluate.py	Scores extraction, findings, recommendations, and safety.

Path	Responsibility
docs/lab-guide.md	Original capstone implementation guide.
extensions/	Extension requirements and acceptance criteria.
tests/test_extensions.py	Focused regression tests for F1, F5, and duplicate behavior.

7. End-to-End Summary

Evidence enters through a sample folder or the upload API. The pipeline classifies files, extracts document fields, assesses photographs, reads optional claims history, runs deterministic validations, builds verification items, checks duplicate identity during API processing, and asks the grounded agent for a policy-aware review. The result is stored durably and exposed to the React workspace. A handler verifies important fields, reviews evidence and findings, records a named outcome, edits it when necessary, and archives completed work.

